

Documentacion tecnica del blog Cultura y Desarrollo

```
**Proyecto:** `ccd_blog`  
**Framework:** Astro 7.3.1  
**Salida:** sitio estatico  
**Despliegue:** GitHub Pages  
**Dominio canonico:** `https://blog.culturaydesarrollo.org`
```

1. Proposito

Este documento describe la arquitectura, el flujo de publicacion, las rutas, los componentes y las tareas de mantenimiento del blog. Sirve como referencia para desarrollar, publicar y revisar cambios sin depender de una base de datos propia.

2. Arquitectura general

El proyecto usa Astro con contenido Markdown y genera HTML estatico durante el build.

```
```text  
src/content/blog/ Articulos Markdown
src/content/autores/ Biografias reutilizables
src/pages/ Rutas estaticas y endpoints XML
src/components/ Componentes reutilizables
src/layouts/ Layout general y metadatos
src/styles/ Estilos globales
public/images/ Imagenes publicas
public/videos/ Videos locales opcionales
```
```

Flujo de compilacion:

1. Astro lee las colecciones `blog` y `autores`.
2. El schema de contenido valida el frontmatter.
3. Las paginas se generan desde las entradas Markdown.
4. Astro crea el directorio `dist/`.
5. GitHub Actions publica `dist/` en GitHub Pages.

No existe un servidor de aplicacion ni una base de datos dentro del repositorio.

3. Requisitos locales

- Node.js 22 o superior.
- npm.
- Git.
- Acceso al repositorio y al proyecto de GitHub Pages.

Comandos principales:

```
```bash  
npm install
npm run dev
npm run typecheck
npm run build
npm run validate
```
```

`npm run validate` ejecuta el typecheck y el build de produccion.

4. Colecciones de contenido

Coleccion `blog`

Definida en `src/content.config.ts`.

Campos principales:

| Campo | Obligatorio | Descripcion |
|---------------|-------------|--|
| --- | --- | --- |
| `title` | Si | Titulo del articulo |
| `description` | Si | Resumen para portada y SEO |
| `pubDate` | Si | Fecha ISO, por ejemplo `2026-09-10` |
| `image` | No | Imagen destacada validada por Astro |
| `author` | Si | Nombre visible del autor |
| `authorSlug` | No | Identificador de la coleccion de autores |
| `authorImage` | No | Imagen del autor; usa una imagen por defecto |
| `authorBio` | No | Respaldo cuando no existe una entrada de autor |
| `featured` | No | Marca editorial para mostrar el articulo destacado |
| `category` | Si | Lista de categorias |
| `tags` | No | Lista de etiquetas |

Ejemplo minimo:

```
```yaml

title: "Titulo del articulo"
description: "Resumen claro y util para la portada."
pubDate: "2026-09-10"
author: "Nombre del autor"
authorSlug: "nombre-del-autor"
authorImage: "/images/nombre-del-autor.jpg"
category:
 - "Cultura"
tags: ["Colombia", "Comunidad"]

```

### ### Coleccion `autores`

Cada archivo en `src/content/autores/` contiene la biografia reutilizable del autor.

```
```yaml
---
name: "Nombre del autor"
slug: "nombre-del-autor"
image: "/images/nombre-del-autor.jpg"
---
```

Biografia editorial del autor.

```

El valor de `authorSlug` debe coincidir con el identificador del archivo. No repetir biografias largas dentro de cada post.

## ## 5. Rutas generadas

- `/` - portada con destacados, busqueda y filtro de autor.

- ``/<slug>/`` - artículo individual.
- ``/categorias/`` - índice de categorías.
- ``/categorias/<categoria>/`` - artículos de una categoría.
- ``/tags/<tag>/`` - artículos de una etiqueta.
- ``/rss.xml`` - feed RSS.
- ``/sitemap.xml`` - sitemap XML.
- ``/robots.txt`` - reglas para rastreadores.

Los slugs se derivan del nombre del archivo Markdown y se normalizan con ``src/lib/slugify.ts`` cuando corresponde.

### ## 6. Portada y búsqueda

La portada muestra en cada artículo el título, descripción, categoría, fecha, nombre y fotografía del autor. La fotografía abre un modal con la biografía.

La búsqueda libre revisa:

- título;
- descripción;
- cuerpo del artículo;
- categorías;
- etiquetas.

El autor no forma parte de la búsqueda textual. Se selecciona mediante el filtro dedicado de autores.

Los artículos con ``featured: true`` se muestran en la zona de destacados. Se recomienda mantener solo dos o tres piezas prioritarias.

### ## 7. Formularios y comentarios

``src/components/ContactForm.astro`` usa Formspree y cambia su encabezado según el contexto:

- Portada: ``Ponte en contacto con nosotros``.
- Artículo: ``Escribenos sobre este artículo``.

El formulario incluye nombre, correo, mensaje, honeypot, validación local y aviso de privacidad. El mensaje requiere al menos 20 caracteres.

Al final del contenido del artículo aparece ``Comenta este artículo...``. El enlace desplaza al formulario contextual mediante ``#contacto``.

En artículos, el formulario envía una referencia con este formato:

```
```text
Referencia: titulo · autor · fecha · Valoracion: ★★★★★ (4/5)
```
```

La valoración se agrega cuando el lector selecciona estrellas. Si no selecciona ninguna, el formulario envía ``Sin valoración``.

### ## 8. Valoración individual

El artículo ofrece cinco botones de estrellas. La selección:

- se guarda en ``localStorage`` usando una clave por artículo;
- actualiza el estado visual de las estrellas;

- actualiza el texto de confirmacion;
- actualiza la referencia visible del formulario;
- se envia junto con el formulario si el lector escribe un mensaje.

La valoracion no es una estadistica global. Actualmente no se muestran vistas, cantidad global de comentarios ni promedio agregado. No agregar cifras inventadas.

## ## 9. Videos

El contenido Markdown acepta HTML para videos locales y externos. Los estilos de `src/pages/[slug].astro` adaptan ambos formatos a pantallas pequenas.

Video local:

```
```html
<figure class="article-video">
  <video controls preload="metadata" poster="/images/portada-video.jpg">
    <source src="/videos/nombre-del-video.mp4" type="video/mp4" />
    Tu navegador no admite la reproduccion de video.
  </video>
  <figcaption>Descripcion del video.</figcaption>
</figure>
```
```

Video externo:

```
```html
<figure class="article-video">
  <iframe
    src="https://www.youtube.com/embed/ID_DEL_VIDEO"
    title="Titulo descriptivo del video"
    loading="lazy"
    allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture"
    allowfullscreen
  ></iframe>
  <figcaption>Descripcion del video.</figcaption>
</figure>
```
```

Los videos locales deben guardarse en `public/videos/`. Se recomienda no activar autoplay, incluir controles, optimizar el peso y comprobar la reproduccion en movil.

## ## 10. SEO y feeds

El layout y `SeoHead.astro` generan title, description, canonical, Open Graph, Twitter Cards y metadatos de articulo. Las rutas XML se generan desde las colecciones y deben revisarse despues de cambios estructurales.

El feed principal es RSS XML por compatibilidad con lectores y agregadores. JSON Feed queda como posible complemento futuro.

## ## 11. Despliegue en GitHub Pages

El workflow `.github/workflows/deploy.yml` se ejecuta en pushes a `main` o `master`:

1. instala Node.js 22;

2. ejecuta `npm ci`;
3. limpia cache de Astro;
4. ejecuta `npm run validate`;
5. crea `CNAME` con el dominio institucional;
6. sube `dist/` como artefacto;
7. despliega mediante GitHub Pages.

El sitio debe conservar una arquitectura estatica. La analitica futura debe usar un proveedor externo alojado, sin endpoints ni base de datos propios en este repositorio.

### ## 12. Procedimiento de publicacion

1. Crear o revisar el archivo Markdown en `src/content/blog/`.
2. Verificar frontmatter, fecha ISO, autor, categorias y etiquetas.
3. Confirmar que la bio y la imagen del autor existen.
4. Optimizar imagenes y videos.
5. Ejecutar `npm run validate`.
6. Revisar la portada, el articulo, el modal del autor, el formulario, la valoracion y los enlaces.
7. Revisar el diff en GitHub Desktop.
8. Crear el commit.
9. Hacer push y revisar el workflow de GitHub Actions.

### ## 13. Mantenimiento y limites conocidos

- Las valoraciones no se agregan en una base global.
- Las vistas y comentarios no se muestran hasta integrar analitica real.
- Formspree gestiona la entrega del correo; cualquier cambio de proveedor requiere actualizar el formulario y la politica de privacidad.
- GitHub Pages no ejecuta endpoints del servidor del proyecto.
- Los archivos generados en `.astro/` siguen la convencion actual del repositorio y deben revisarse antes de confirmar cambios.

### ## 14. Verificacion posterior al despliegue

Comprobar manualmente:

- portada y filtro de autor;
- busqueda por titulo, contenido, categoria y etiqueta;
- apertura del modal del autor;
- navegacion de categorias y tags;
- enlace `Comenta este articulo...`;
- actualizacion visible de estrellas y referencia del formulario;
- validacion y envio del formulario;
- reproduccion de videos locales o externos;
- `/rss.xml`, `/sitemap.xml` y `/robots.txt`;
- workflow de GitHub Pages.